

Smart MCQ Solver Challenge

1. Abstract / Executive Summary

The objective of this project was to develop an intelligent Multiple Choice Question (MCQ) answering system for the Smart MCQ Solver Challenge. Given a question prompt and five answer options, the model predicts and ranks the most probable answers. The competition evaluates submissions using the Mean Average Precision at 3 (MAP@3) metric, which measures the quality of the top three ranked predictions.

The project began with an Exploratory Data Analysis (EDA) to understand the dataset, examine the distribution of answer classes, identify missing values, and inspect the overall data quality. The textual data from the question prompts and answer options was then preprocessed through lowercasing, punctuation removal, and tokenization to create a clean corpus for training.

Word2Vec was used to learn dense semantic word embeddings from the combined corpus of prompts and answer options. These embeddings were used to represent both the question prompt and each candidate answer. A custom neural network architecture consisting of a **QAScorer** module and a **MultipleChoiceModel** was developed to estimate the compatibility score between the prompt and each answer option. The model generated a score for every option, and these scores were used to rank the candidate answers.

The model was implemented using PyTorch and trained on the competition dataset. Performance was evaluated using the MAP@3 metric, which aligns with the competition's evaluation criterion and rewards correct answers appearing higher in the predicted ranking.

This project provided practical experience in natural language preprocessing, word embedding techniques, neural network design, multiple-choice answer ranking, and model evaluation. It also demonstrated how distributed word representations and deep learning can be combined to solve real-world NLP ranking tasks.

2. Introduction

2.1 Problem Statement

The Smart MCQ Solver Challenge is a Natural Language Processing (NLP) task that focuses on automatically answering multiple-choice questions. Each sample in the dataset consists of a question prompt and five candidate answer options (A, B, C, D, and E). The objective is to build a model capable of understanding the semantic relationship between the question and its answer choices, ranking the options according to their relevance, and identifying the correct answer. Unlike traditional classification tasks, the competition evaluates the quality of the top three predicted answers using the Mean Average Precision at 3 (MAP@3) metric.

The primary challenge is to learn meaningful semantic representations from the textual data and accurately rank the candidate answers for unseen questions.

2.2 Project Objective

The primary objective of this project was to design and implement an end-to-end NLP pipeline for solving the Smart MCQ Solver Challenge. The specific goals of the project were:

- To perform Exploratory Data Analysis (EDA) for understanding the dataset and identifying important characteristics.
- To preprocess textual data through cleaning and tokenization.
- To train Word2Vec embeddings for learning semantic representations of words.
- To design a custom neural network using PyTorch that scores the compatibility between a question prompt and each answer option.
- To evaluate the model using the MAP@3 metric, which is the official evaluation metric of the competition.
- To strengthen practical skills in PyTorch, neural network implementation, word embeddings, and NLP-based ranking models.

2.3 Report Structure

The remainder of this report is organized as follows:

- **Section 3** presents the Exploratory Data Analysis (EDA), including dataset characteristics and key observations.
- **Section 4** describes the data preprocessing pipeline, including text cleaning and tokenization.
- **Section 5** explains the Word2Vec embedding model and the custom neural network architecture developed for scoring answer options.
- **Section 6** discusses the training procedure, model configuration, and implementation details.
- **Section 7** presents the evaluation methodology, experimental results, and performance analysis using the MAP@3 metric.
- **Section 8** summarizes the conclusions, key findings, limitations, and possible future improvements.

3. Dataset & Preprocessing

The dataset consists of training and test samples, where each instance contains a question prompt and five answer options (A–E). An initial Exploratory Data Analysis (EDA) showed that the dataset contains no missing values and only a moderate class imbalance, with option **B** being the most frequent and **E** the least frequent. Text preprocessing involved converting all text to lowercase, removing punctuation, and tokenizing the cleaned text into word-level tokens. The vocabulary was then analyzed and used to train Word2Vec embeddings for semantic representation. No additional data augmentation techniques were applied, as the original dataset was sufficient for model training.

4. Tokenization Strategy

The input text was converted into model-readable tokens using the Hugging Face tokenization pipeline. Each training example was formed by combining the question prompt with one answer option, allowing the model to learn the semantic relationship between them.

Primary Tokenizer Selection:

The best-performing model uses the **DeBERTa-v3-small tokenizer**

([microsoft/deberta-v3-small](#)). This tokenizer is associated with the DeBERTa-v3 pre-trained transformer model and employs a subword tokenization strategy to efficiently represent words, including rare and unseen terms.

Justification:

Subword tokenization preserves contextual information while reducing the out-of-vocabulary problem. Since DeBERTa is designed for contextual language understanding, this strategy is well suited for multiple-choice question answering, where capturing semantic relationships between the prompt and answer options is essential.

Implementation Details:

The tokenization pipeline was implemented using Hugging Face's **AutoTokenizer**. Tokenization was performed in batches with `padding="max_length"`, `truncation=True`, and a maximum sequence length of **128** tokens. The tokenizer generated `input_ids` and `attention_mask`, which were used as inputs to the model during training and inference.

Experimental Comparison:

An earlier baseline model used **Word2Vec** with word-level tokenization to generate static word embeddings. Although this approach provided a simple semantic baseline, the contextual tokenization and representations learned by DeBERTa produced better performance, making it the final choice for the project.

5. Modeling & Experimentation

5.1 Model 1: Fine-Tuned Transformer Model (DeBERTa-v3-small)

Architecture:

The primary model used in this project is **DeBERTa-v3-small** ([microsoft/deberta-v3-small](#)), a pre-trained Transformer encoder available through the Hugging Face Transformers library.

Salient Points:

The model uses a disentangled self-attention mechanism to capture contextual relationships between the question prompt and answer options, making it well suited for multiple-choice question answering.

Fine-Tuning Strategy:

The model was fine-tuned using the **AdamW** optimizer with a learning rate scheduler, gradient accumulation, and cross-entropy loss. Training was performed for multiple epochs to optimize performance on the competition dataset.

Quantization:

No quantization techniques were applied.

5.2 Model 2: Custom Deep Learning Model (Word2Vec + PyTorch)

Architecture:

A custom neural network was developed using **Word2Vec** embeddings and PyTorch. Word embeddings were generated from the training corpus and passed to a custom **QAScorer** network that computed compatibility scores between the question and each answer option.

Salient Points:

This lightweight architecture demonstrated how semantic word embeddings and a custom neural network can effectively perform answer ranking without relying on large pre-trained language models.

5.3 Model 3: Fine-Tuned Transformer Model (DeBERTa-v3-base)

Architecture:

The third model uses **DeBERTa-v3-base**, a larger version of the DeBERTa architecture with increased model capacity and richer contextual representations compared to the small variant.

Salient Points:

The larger encoder captures more complex semantic relationships between the prompt and answer choices, resulting in improved prediction quality. The model was fine-tuned using the Hugging Face Transformers framework with the same tokenization pipeline and training strategy as the smaller model, making it suitable for achieving higher competition performance.

6. Performance & Comparative Analysis

6.1 Evaluation Metrics

The primary evaluation metric used in the Kaggle competition was **Mean Average Precision at 3 (MAP@3)**, which measures whether the correct answer appears within the top three predictions while rewarding higher-ranked correct answers. During training, additional metrics such as **Accuracy**, **Cross-Entropy Loss**, and **F1-score** were monitored to evaluate model performance.

6.2 Training Performance

Training and validation performance for all models were monitored using **Weights & Biases (W&B)**. Loss and accuracy curves were analyzed to ensure stable convergence and detect potential overfitting. The corresponding W&B visualizations are included below for reference.

6.3 Comparative Report

Model	MAP@3	Remarks
Word2Vec + PyTorch	0.75	Lightweight baseline model
DeBERTa-v3-small	0.99	Better contextual understanding and improved performance
DeBERTa-v3-base	0.97	good overall performance with lower computational cost

The transformer-based models significantly outperformed the custom Word2Vec model due to their ability to learn contextual representations. Among them, **DeBERTa-v3-small** achieved the best predictive performance, although it required greater training time and computational resources.

6.4 Kaggle Performance

The final ensemble model achieved a **Kaggle MAP@3 score of 0.75** and secured a leaderboard rank of **663**. This demonstrates the effectiveness of combining transformer-based models for the multiple-choice question answering task.

7. Conclusion & Future Work

7.1 Key Learnings

This project provided hands-on experience in building end-to-end NLP pipelines, including data preprocessing, tokenization, custom neural network development, and fine-tuning transformer models using PyTorch and Hugging Face. It also improved my understanding of model evaluation, experiment tracking with Weights & Biases, and the practical application of MAP@3 for ranking tasks.

7.2 Challenges Faced

The major challenges included selecting an effective model architecture, tuning hyperparameters, and balancing model performance with computational resources. Multiple experiments were conducted to compare different approaches, leading to the selection of the best-performing model.

7.3 Areas for Improvement

Future work could focus on experimenting with larger ensembles, advanced hyperparameter optimization, and more efficient inference techniques such as Test Time Augmentation (TTA). Although both **DeBERTa-v3-small** and **DeBERTa-v3-base** were evaluated, **DeBERTa-v3-small** achieved the best overall performance on this dataset, indicating that a smaller, well-tuned model can outperform a larger model for this specific task.

8. References

1. Kaggle. *Smart MCQ Solver Challenge*. <https://www.kaggle.com/competitions/smart-mcq-solver-challenge>
2. Hugging Face. *DeBERTa-v3-small Model Card*. <https://huggingface.co/microsoft/deberta-v3-small>
3. Hugging Face. *Transformers Documentation*. <https://huggingface.co/docs/transformers>
4. PyTorch Team. *PyTorch Documentation*. <https://pytorch.org/docs/stable/>
5. Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). *Efficient Estimation of Word Representations in Vector Space*. <https://arxiv.org/abs/1301.3781>
6. He, P., Liu, X., Gao, J., & Chen, W. (2021). *DeBERTa: Decoding-enhanced BERT with Disentangled Attention*. <https://arxiv.org/abs/2006.03654>
7. Weights & Biases. *Experiment Tracking Documentation*. <https://docs.wandb.ai/>
8. Pedregosa, F., et al. (2011). *Scikit-learn: Machine Learning in Python*. *Journal of Machine Learning Research*, 12, 2825–2830. <https://scikit-learn.org/>
9. Reimers, N., & Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. <https://arxiv.org/abs/1908.10084>